

Contents

Security Review with Fix Plan — StudioBook (fictional AI-built class-booking app)	1
Scope & method	1
Risk summary	1
Findings	1
B-01 — Critical	1
B-02 — Critical	2
B-03 — Critical	3
B-04 — Critical	4
B-05 — High	4
B-06 — High	5
B-07 — High	5
B-08 — Medium	6
B-09 — High	7
B-10 — Medium	8
B-11 — Medium	8
B-12 — Low	9
Remediation plan	10
Retest notes	10

Security Review with Fix Plan — StudioBook (fictional AI-built class-booking app)

Deliberately insecure for demonstration and training. Do not deploy. Run locally only.

Prepared: 2026-09-28 · **Tier:** Standard/Advanced · **Target:** `targets/vibe-app` (baseline)
vs. `targets/vibe-app-fixed` (retest target)

Scope & method

Same scope as `RISK_SCAN_TOP10.md`, extended to all 12 findings, with the differentiator this listing promises: an **AI fix prompt** per finding, parsed directly from `targets/vibe-app-fixed/AI_FIX_PROMPTS.md` (the file written before any fix, then actually used to build this repo's own fixed branch) - not re-typed into this report, so it can't silently drift from what was really run. B-11 has no separate prompt (its fix landed inside the B-04 commit - see that entry below); its excerpt says so rather than inventing one.

Risk summary

Severity	Count
Critical	4
High	4
Medium	3
Low	1
Total	12

Findings

B-01 — Critical

OWASP: A05:2021 - Security Misconfiguration · API8:2023 - Security Misconfiguration

Effort to fix: S

How found: tool (gitleaks, semgrep)

Evidence:

- `.env:20`
- `components/AdminUserList.tsx:7`

What's wrong. The Supabase service-role key (bypasses every RLS policy in the database) is read from a `NEXT_PUBLIC_`-prefixed environment variable inside a client component.

Why it matters. Next.js inlines every `NEXT_PUBLIC_*` variable into the JavaScript bundle shipped to every visitor's browser, logged in or not. Anyone who views page source gets a credential that can read or write ANY row in the database, unrestricted.

Exact fix. Move the query behind a server-only route using the service-role client correctly (never in a "use client" component); delete the `NEXT_PUBLIC_` variable entirely. Diff: `git show 1ca4b02`.

AI fix prompt (copy-paste into Claude Code / Cursor):

```
In `targets/vibe-app-fixed/components/AdminUserList.tsx`, stop building a
Supabase client directly in this `"use client"` component from a
`NEXT_PUBLIC_`-prefixed service-role var. Move the user-listing query into a
new server-only module or route that uses the existing
`lib/supabase/admin.ts` `createAdminClient()` (already `server-only`-guarded),
and have this component fetch from that route instead. Remove
`NEXT_PUBLIC_SUPABASE_SERVICE_ROLE_KEY` from `.env`/`.env.example` entirely -
only the non-prefixed `SUPABASE_SERVICE_ROLE_KEY` should exist. Acceptance
check: `npm run build` in `targets/vibe-app-fixed`, then `grep -r
"service_role" .next/static` finds nothing.
```

B-02 — Critical

OWASP: A02:2021 - Cryptographic Failures · API8:2023 - Security Misconfiguration

Effort to fix: S

How found: tool (gitleaks)

Evidence:

- `.env:3`
- `lib/stripe.ts:3`

What's wrong. A Stripe secret key is hardcoded directly in a server file, and `.env` (holding the same value plus other credentials) is committed to the repository.

Why it matters. Anyone with read access to the repository - a contractor, a leaked backup, an over-shared CI log - obtains a live-shaped credential that can act as the application against Stripe's API.

Exact fix. Read the secret from an environment variable with no hardcoded fallback (fail to start if unset); add `.env` to `.gitignore` and stop committing it. Diff: `git show 9026039`, `git show 9c39b99` (a follow-up commit after the first fix was found incomplete).

AI fix prompt (copy-paste into Claude Code / Cursor):

```
In `targets/vibe-app-fixed/lib/stripe.ts`, remove the hardcoded
`STRIPE_SECRET_KEY` literal. Read it from `process.env.STRIPE_SECRET_KEY` with
no fallback string, and throw at import time if it's unset, so the app fails
to start rather than silently running with no key. Add `.env` to
`targets/vibe-app-fixed/.gitignore` (remove the header comment there that says
it deliberately isn't listed) and delete the committed `targets/vibe-app-fixed/.env`
from the repo - `.env.example` is the only committed template from here on.
Do not change `targets/vibe-app/` (the baseline must keep this flaw).
Acceptance check: `git ls-files targets/vibe-app-fixed | grep -x
```

targets/vibe-app-fixed/.env` finds nothing; `gitleaks git --config scanners/gitleaks/gitleaks.toml` reports no hit under `targets/vibe-app-fixed/`.

Deviation from the prompt above, caught in review: B-02's acceptance check ("gitleaks git reports no hit under targets/vibe-app-fixed/") is not met by git HISTORY, only by the current tree: this module's very first commit was a verbatim copy of the baseline, fake secrets included, before this fix landed two commits later - gitleaks git (full-history mode) still finds that historical diff every time, by design. See this module's own README.md ("Known trade-off") for why that's expected and not a live finding.

B-03 — Critical

OWASP: A01:2021 - Broken Access Control · API1:2023 - Broken Object Level Authorization

Effort to fix: M

How found: tool (rls-checker)

Evidence:

- supabase/migrations/20240101000002_rls_and_storage.sql:39

What's wrong. Row-level security is never enabled on the bookings table at all.

Why it matters. Any authenticated user can read, update, or delete EVERY other user's bookings directly through Supabase's own REST API, completely bypassing the Next.js app's own routes and any checks they perform.

Exact fix. Enable RLS with an owner/admin SELECT policy; explicitly revoke INSERT/UPDATE/DELETE from `authenticated` - every write must go through a service-role route or RPC, since a policy alone doesn't override Supabase's default table-level grants. Diff: `git show 40c4128`.

AI fix prompt (copy-paste into Claude Code / Cursor):

```
Add a new migration `targets/vibe-app-fixed/supabase/migrations/*_fix_rls.sql`  
(don't edit the baseline migrations) that enables RLS on `public.bookings` and  
adds a SELECT policy scoped to `user_id = auth.uid() OR public.is_admin()`  
(create `public.is_admin()` as a `SECURITY DEFINER`, `STABLE`, `SET  
search_path = ''` function reading `profiles.role` for `auth.uid()`, revoked  
from `anon`/`authenticated`/`public` and granted only to itself being callable  
via RLS - a plain subquery on `profiles` inside a `profiles` policy would  
recurse). Revoke `INSERT`, `UPDATE`, `DELETE` on `bookings` from  
`authenticated` - all writes happen server-side via the service-role client  
or an RPC (see B-10), so a direct PostgREST write must be denied outright,  
not just RLS-narrowed. Acceptance check: `scanners/rls-checker/rls_checker.py`  
against this stack's DB reports 0 FAIL for `bookings`.
```

Deviation from the prompt above, caught in review: B-03's prompt was itself **wrong**, and the commit did not follow it: it says to create `is_admin()` "revoked from `anon/authenticated/public` and granted only to itself being callable via RLS". That's a technical error - Postgres evaluates an RLS policy expression AS the connecting role, so revoking EXECUTE on a function a policy calls makes the policy fail for every real caller, not just lock the function down (the opposite of what B-06/B-10's RPCs need, where EXECUTE genuinely should be service_role-only because those trust their caller completely instead of checking `auth.uid()` themselves). The commit that actually landed kept EXECUTE granted to `anon/authenticated`, correctly. A later attempt to "fix" this by silently rewriting the prompt text itself (rather than recording it here) was caught in review and reverted - the prompt above is the original, unedited, technically-wrong version that was really run.

B-04 — Critical

OWASP: A01:2021 - Broken Access Control · API3:2023 - Broken Object Property Level Authorization

Effort to fix: S

How found: tool (rls-checker)

Evidence:

- supabase/migrations/20240101000002_rls_and_storage.sql:10

What's wrong. RLS IS enabled on profiles, but the SELECT policy is using `(true)` - equivalent to no policy at all for reads.

Why it matters. Every authenticated user can read every other user's profile row, including email and phone number - a full customer-PII leak.

Exact fix. Replace the policy with `id = auth.uid() OR is_admin()`; add a SECURITY DEFINER `is_admin()` helper to avoid the self-referential-policy recursion a plain subquery on profiles would hit. Diff: `git show 40c4128`.

AI fix prompt (copy-paste into Claude Code / Cursor):

In the same new migration as B-03 (or a second one), replace ``profiles_select_any_row_b04`'s `using (true)`` with ``using (id = auth.uid() OR public.is_admin())``. This will make two existing read paths silently return empty instead of erroring - fix both in this same commit so nothing is left broken: ``app/studio/members/page.tsx`` (B-11's page - do the full B-11 fix here: read server-side with the admin client after an admin check, select only ``id, full_name``) and ``app/studio/dashboard/page.tsx``'s member/booking queries (also read via the admin client, since it's an admin-only page). Acceptance check: ``scanners/rls-checker/rls_checker.py`` reports 0 FAIL for ``profiles``; ``/studio/members`` and ``/studio/dashboard`` still render real data when visited as ``noah.admin``.

Deviation from the prompt above, caught in review: B-04's prompt says the compensating fix to `app/studio/members/page.tsx` should "read server-side with the admin client after an admin check". The page that actually landed is **auth-only** (any signed-in member can view it, matching the baseline's own access scope for this page/B-11's original finding) - it uses the service-role client to read `{id, full_name}` because B-04's new RLS policy would otherwise only let a member see their own row, not because the page is admin-restricted. It isn't.

B-05 — High

OWASP: A01:2021 - Broken Access Control · API1:2023 - Broken Object Level Authorization

Effort to fix: S

How found: tool (semgrep)

Evidence:

- app/api/bookings/[id]/cancel/route.ts:5

What's wrong. POST `/api/bookings/{id}/cancel` checks that the caller is logged in, but never checks that the booking being cancelled belongs to them.

Why it matters. Any authenticated user can cancel any OTHER user's booking by id - a denial-of-service against a specific victim's reservations.

Exact fix. Filter the update on `id AND user_id` (AND status, to make a second cancel a no-op); return the same 404 whether the booking belongs to someone else or doesn't exist. Diff: `git show 0a17f5c`.

AI fix prompt (copy-paste into Claude Code / Cursor):

In `app/api/bookings/[id]/cancel/route.ts`, validate `params.id` is a UUID (400 if not), then update via the service-role client with `.eq("id", id).eq("user_id", user.id).eq("status", "confirmed")` - return 404` if no row was affected, whether the booking belongs to someone else or doesn't exist (don't let the response distinguish the two). Acceptance check: `exploits/B-05-cancel-any-booking.sh` (run against this fixed stack) shows liam getting a 404 and maya's booking still `confirmed`.

Deviation from the prompt above, caught in review: B-05's prompt says an invalid `params.id` should get a 400. The route that landed returns **404** for that case too, for the same reason the prompt itself gives for not distinguishing "someone else's booking" from "doesn't exist" - an invalid id is just another shape of "not found", not a separately-observable response.

B-06 — High

OWASP: A01:2021 - Broken Access Control · API5:2023 - Broken Function Level Authorization

Effort to fix: S

How found: manual

Evidence:

- `app/admin/page.tsx:6`
- `app/api/admin/grant-credits/route.ts:6`

What's wrong. The admin-only 'grant credits' page hides its form from non-admins in the UI, but the route behind it never re-checks the caller's role server-side.

Why it matters. Any authenticated user can call the route directly and grant themselves - or anyone - unlimited free credits, bypassing the studio's entire paid-credit model.

Exact fix. Add a shared `requireAdmin()` helper (reads `profiles.role` via the service-role client, never client-supplied data) and call it before doing anything else in the route, returning 403 before even looking up the target user. Diff: `git show a66f945`.

AI fix prompt (copy-paste into Claude Code / Cursor):

Add a `requireAdmin()` helper (e.g. `lib/auth/require-admin.ts`) that calls `getSessionUser()`, then looks up `profiles.role` for that user's id via the service-role client (never trust `user_metadata`, request body, or unverified JWT claims), returning the profile or `null`. In `app/api/admin/grant-credits/route.ts`, call it and return 403 *before* looking up the target user if the caller isn't an admin. Validate `amount` is a bounded positive integer and `targetUserId` is a UUID; use an atomic `credit_balance = credit_balance + amount` update, not read-then-write. Reuse the same helper in `/admin` and `/studio/dashboard`'s page-level checks. Acceptance check: `exploits/B-06-grant-credits-without-admin-role.sh` against this stack returns 403 with liam's balance unchanged.

B-07 — High

OWASP: A08:2021 - Software and Data Integrity Failures · API8:2023 - Security Misconfiguration

Effort to fix: S

How found: manual

Evidence:

- `app/api/stripe/webhook/route.ts:5`

What's wrong. The Stripe webhook handler parses the request body directly with no signature verification at all.

Why it matters. Anyone who knows the webhook URL can POST an arbitrary forged event and have it processed as if Stripe sent it - including a fake `checkout.session.completed` that grants free credits (see B-08).

Exact fix. Verify the raw body against the `stripe-signature` header with `stripe.webhooks.constructEvent()` before parsing anything; fail closed if the webhook secret is unset. Diff: `git show 5e30c6f`, `git show 9c39b99` (a follow-up commit after the first fix was found incomplete).

AI fix prompt (copy-paste into Claude Code / Cursor):

```
In `app/api/stripe/webhook/route.ts`, read the raw body with
`await request.text()` (not `.json()`), and call
`stripe.webhooks.constructEvent(rawBody, request.headers.get("stripe-signature"),
process.env.STRIPE_WEBHOOK_SECRET)` inside a try/catch, returning 400 on any
failure (missing header, wrong secret, tampered body) without touching the
database. Fail closed (500) at import/request time if
`STRIPE_WEBHOOK_SECRET` is unset. Acceptance check:
`exploits/B-07-webhook-no-signature-check.sh` against this stack gets HTTP
400 for the unsigned forged event.
```

B-08 — Medium

OWASP: A04:2021 - Insecure Design · API8:2023 - Security Misconfiguration

Effort to fix: M

How found: manual

Evidence:

- `app/api/stripe/webhook/route.ts:11`

What's wrong. The webhook handler has no dedup against the Stripe event's own id - POSTing the identical event body twice grants credits twice.

Why it matters. A replayed (or maliciously resubmitted) webhook event grants the same credits repeatedly - direct financial loss for the business, scaling with however many times the event is replayed.

Exact fix. Record processed event ids in a table with a unique constraint, and grant credits in the SAME atomic statement (a SECURITY DEFINER RPC) so there is no window where one succeeded without the other. Diff: `git show 292563a`, `git show 9c39b99` (a follow-up commit after the first fix was found incomplete).

AI fix prompt (copy-paste into Claude Code / Cursor):

```
Add a migration creating `stripe_events(event_id text primary key,
checkout_session_id text unique not null, processed_at timestamptz not null
default now())` (RLS enabled, zero policies, all grants to
anon/authenticated revoked - service_role only) and a `SECURITY DEFINER`
function `apply_checkout_credits(p_event_id text, p_session_id text,
p_user_id uuid, p_credits int) returns text` that inserts into
`stripe_events` with `on conflict (event_id) do nothing returning event_id`;
if no row came back, return `duplicate`; otherwise atomically run `update
profiles set credit_balance = credit_balance + p_credits where id =
p_user_id` and return `granted`. Revoke EXECUTE on this function from
`public`/`anon`/`authenticated`; grant only to `service_role` (Supabase
grants EXECUTE on every new function to `anon`/`authenticated` by default -
this must be explicitly revoked or the function becomes a public,
unauthenticated endpoint). In the webhook route (after B-07's signature
check), require `session.payment_status === "paid"`, validate `credits` is a
positive integer from trusted metadata, and call this RPC via the
```

service-role client instead of the old select-then-update. Acceptance check: ``exploits/B-08-webhook-duplicate-event-double-grant.sh``, updated to send a correctly-signed event (via ``stripe.webhooks.generateTestHeaderString``) twice, shows the balance moving by the credit amount exactly once.

Deviation from the prompt above, caught in review: B-08 needed a follow-up fix, not just its original prompt. The RPC as first built deduped only on an exact repeated `event_id`; a milestone review found that a *different* event id for the *same* checkout session (a real shape Stripe redelivery can take) still double-granted, because the `on conflict (event_id) target` didn't cover the table's other unique constraint. Fixed in a follow-up migration (`20240101000008_fix_b08_conflict_target.sql`) changing it to a bare `on conflict do nothing`, which catches a violation of either constraint. `exploits/B-08-webhook-duplicate-event-double-grant.sh` originally only replayed the identical event id, so it would not have caught this by itself - a second check (two differently-id'd events for the same checkout session) was added to the script as a regression test.

B-09 — High

OWASP: A03:2021 - Injection · API8:2023 - Security Misconfiguration

Effort to fix: S

How found: manual

Evidence:

- `components/AvatarUpload.tsx:7`
- `supabase/migrations/20240101000002_rls_and_storage.sql:50`

What's wrong. Avatar upload accepts any file type and size to a public storage bucket; an uploaded SVG with an embedded

is served back as `image/svg+xml` with no attachment disposition.

Why it matters. Stored XSS - a malicious SVG avatar executes its script in the browser of anyone who views it, including via the public bucket URL directly.

Exact fix. Restrict the bucket's allowed `_mime_types` to a real-image allow-list (no SVG) with a file size cap, and derive the stored filename's extension from the validated type, never the raw uploaded filename. Diff: `git show 87dd892`.

AI fix prompt (copy-paste into Claude Code / Cursor):

Add a migration that ``update`s` (not re-``insert`s` - the existing ``insert ... on conflict do nothing`` would no-op against the already-seeded row) the ``avatars`` bucket to set ``file_size_limit`` (e.g. 2 MiB) and ``allowed_mime_types`` to ``{image/png,image/jpeg,image/webp}`` (no ``image/svg+xml``). In ``components/AvatarUpload.tsx``, generate the storage path server-side from the authenticated user's id and a fixed extension derived from the validated MIME type (never from ``file.name``). Acceptance check: ``exploits/B-09-svg-avatar-stored-xss.sh`` against this stack gets a non-2xx upload response for the SVG.

Deviation from the prompt above, caught in review: B-09's prompt says to “generate the storage path server-side”. The upload that landed builds the path **client-side**, in `components/AvatarUpload.tsx`, from the MIME type validated against the bucket's own `allowed_mime_types` - the real, enforced security boundary is the bucket configuration (Supabase Storage rejects the request server-side regardless of what path the client asks for), not which process happens to concatenate the path string.

B-10 — Medium

OWASP: A04:2021 - Insecure Design · API4:2023 - Unrestricted Resource Consumption

Effort to fix: M

How found: manual

Evidence:

- `app/api/bookings/route.ts:20`

What's wrong. The booking route accepts `quantity` with no server-side validation - a negative value is accepted and, combined with the duplicated pricing formula (B-12), actually INCREASES the caller's credit balance instead of charging them.

Why it matters. Any authenticated user can generate unlimited free credits for themselves with a single crafted request - direct financial loss.

Exact fix. Validate `quantity` is a positive integer within a sane bound before it reaches the database; move the booking + credit debit into one atomic, capacity/balance-checking database function. Diff: `git show 4bc097d`, `git show 9c39b99` (a follow-up commit after the first fix was found incomplete).

AI fix prompt (copy-paste into Claude Code / Cursor):

```
Add a migration with `check (quantity > 0)` on `bookings` and `check
(credit_balance >= 0)` on `profiles`, plus a `SECURITY DEFINER` function
`book_class(p_user_id uuid, p_class_id uuid, p_quantity int, p_cost int)
returns bookings` that: locks the class row (`select ... for update`), checks
remaining capacity against confirmed bookings, checks
`credit_balance >= p_cost`, then atomically decrements the balance and
inserts the booking in one statement, raising an exception (mapped to a 4xx
by the route) if either check fails. Revoke EXECUTE from
`anon`/`authenticated`, grant only to `service_role`. In
`app/api/bookings/route.ts`, validate `quantity` is a positive integer within
a sane upper bound (reject `NaN`, floats, strings, zero, negatives) before
computing cost via the one shared `lib/pricing.ts` function (see B-12), then
call this RPC via the service-role client instead of the old
select-balance/subtract/update/insert sequence. Acceptance check:
`exploits/B-10-negative-quantity-booking.sh` against this stack returns 400
with liam's balance unchanged.
```

B-11 — Medium

OWASP: A01:2021 - Broken Access Control · API3:2023 - Broken Object Property Level Authorization

Effort to fix: S

How found: manual

Evidence:

- `app/studio/members/page.tsx:11`
- `components/MemberNameList.tsx:4`

What's wrong. The studio-members server component fetches (and ships to the browser in the React Server Component payload) every OTHER member's full profile row, including phone number - the page only ever renders their name.

Why it matters. Any member's browser receives every other member's email and phone number in the page's data payload, even though the rendered UI never shows it - trivially recoverable by inspecting network traffic.

Exact fix. Select only the columns the page actually renders (`{id, full_name}`) - fixing what data is FETCHED, not just what's displayed, closes the leak regardless of future rendering changes. Diff: `git show 40c4128`.

AI fix prompt (copy-paste into Claude Code / Cursor):

Covered as part of the B-04 commit above (the fix to ``/studio/members`` had to land in the same commit as the RLS change it depends on, so the page doesn't silently break) - see that section. This entry exists so the finding ID still has its own row in the report; there is no separate commit for it.

B-12 — Low

OWASP: N/A - maintainability, not a security control · N/A - maintainability, not a security control

Effort to fix: M

How found: manual

Evidence:

- `app/api/bookings/route.ts:24`
- `app/page.tsx:5`
- `app/studio/dashboard/page.tsx:4`
- `lib/pricing.ts:1`

What's wrong. The credit-cost pricing formula is duplicated three times (client display, server charge, admin dashboard) with only one copy validating anything; Supabase queries are scattered inline across page components; the dashboard page is ~600 lines mixing data-fetching, aggregation and rendering; there are zero automated tests.

Why it matters. Not directly exploitable on its own, but this is exactly the surface where a fix for one bug (like B-10's quantity clamp) gets applied to one duplicated copy of the logic and silently missed in the others - as it literally did in the baseline.

Exact fix. One shared, VALIDATING pricing function; one data-access module for the dashboard; the page split into small components; tests covering the pricing/validation logic. Diff: `git show 47e926f`, `git show 9c39b99` (a follow-up commit after the first fix was found incomplete).

AI fix prompt (copy-paste into Claude Code / Cursor):

In ``lib/pricing.ts``, change ``estimateCreditCost`` so the `*shared*` function validates (``quantity`` must be a positive integer, throws otherwise) instead of clamping - clamping would silently charge for 1 seat on a negative quantity instead of rejecting it, reintroducing B-10's bug through the back door. Give the UI its own separate display-only clamp for the `*estimate shown before submission*`, clearly named and commented as UI-only, importing the same validating function for the real calculation. Delete the two duplicate inline formulas in ``app/api/bookings/route.ts`` and ``app/studio/dashboard/page.tsx`` (the latter's ``dashboardEstimateCreditCost``) and import from ``lib/pricing.ts`` instead. Split ``app/studio/dashboard/page.tsx`` into a data-loading module (e.g. ``lib/data/dashboard.ts``, one Supabase query per concern) plus several smaller presentational components under ``components/dashboard/``, so no single file mixes fetching, aggregation and rendering at the ~600-line scale the baseline does. Add ``vitest`` tests for ``lib/pricing.ts`` (valid/invalid quantity) and for ``book_class``'s route-level validation. Acceptance check: ``rg "credit_cost\s*\s*"`` matches only inside ``lib/pricing.ts``; ``npm test`` passes; no single file under ``app/`` exceeds ~150 lines.

Deviation from the prompt above, caught in review: B-12's prompt asks for `vitest` tests covering `book_class`'s route-level validation specifically. What exists is `lib/validation.test.ts` (unit tests for the shared validation function) plus live coverage of `book_class` itself via `exploits/B-10-negative-quantity-booking.sh` (quantities `-5`, `0`, `21`, non-integer, and a negative cost passed directly to the RPC) - real coverage of the same cases, but as an integration exploit script against a live database, not a `vitest` test.

Remediation plan

Sprint	Findings	Rationale
Sprint 1	B-01, B-02, B-03, B-04	Critical - active cross-tenant/cross-user data exposure or full compromise
Sprint 2	B-05, B-06, B-07, B-09	High - exploitable with realistic effort, real business impact
Sprint 3	B-08, B-10, B-11	Medium - narrower blast radius or requires a secondary condition
Backlog	B-12	Low - maintainability/hygiene, not directly exploitable

Retest notes

`targets/vibe-app-fixed` is this repo's own retest target - a separate Next.js app and Supabase project implementing every fix above (see its README's "What changed, per finding" table and the per-finding commit references above), built by applying the AI fix prompts above, one commit per finding id (B-03/B-04/B-11 share one commit - see the B-11 entry above for why). Retest evidence, all from committed, generated results files:

- RLS checker vs. the fixed stack: `scanners/results/rls-checker/vibe-app-fixed/rls-matrix.md` - 0 FAIL.
- gitleaks vs. the fixed module's current tree: `scanners/results/gitleaks/vibe-app-fixed-tree.md` - **0 hits in git-tracked files** (any hits are in the local, gitignored `.env` only).
- Semgrep vs. the fixed module's current tree: `scanners/results/semgrep/vibe-app-fixed.json` - **0 findings** (the B-01/B-05 patterns this ruleset targets are both gone).
- Every script in `targets/vibe-app-fixed/exploits/` was run by hand against a freshly-reset stack during development and printed `FIXED` with exit 0 - not yet captured to a committed results file (unlike the three checks above); re-run them yourself to confirm before relying on this line for a real retest sign-off.